

ML Specification Report — home-data-for-ml-course

Ames Housing Sale-Price Regression • AIDE agent (20-step tree search, Claude Sonnet via shim)

Scores grounded exclusively in `facts_aide.json` for run `2-wise-fresh-donkey`; pipeline configuration read from AIDE's own `best_solution.py` and `report.md`, step-level intent from `journal.json`, and task metadata from `competitions/home-data-for-ml-course/config.yaml`.

Contents	
Ames Housing Sale-Price Regression • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
The search trajectory	4
Training configuration	6
Inference procedures	8
Evaluation & Benchmarking	8

Overview

The task is to predict `SalePrice` for each house in the Ames Housing test split — a small, wide, heavily categorical regression problem. This run was executed by **AIDE**, a tree-search coding agent: each step writes one complete `solution.py`, executes it in a sandbox under a 30-minute wall clock, reads back the printed metric or the traceback, and mutates the incumbent. Over **20 steps** it produced **18 scored solutions and 2 crashes**, and its champion — found at **step 18** — is a **four-learner convex-weighted ensemble** (seed-bagged XGBoost + seed-bagged CatBoost + Ridge + ElasticNet, all on a log-transformed target) scoring **local CV RMSLE 0.11031**, and placing **72 / 4103 (98.3rd percentile)** on the public leaderboard.

The two scores above are in different units and must never be compared. AIDE's local metric is RMSE on `log1p(SalePrice)` — the standard RMSLE formulation, a **dimensionless** quantity, and the number the search optimized: **0.11031**. The public leaderboard score recorded in `facts_aide.json` is **12887.15456**, which is on the **raw-dollar scale**. The apparent chasm between 0.11031 and 12887.15456 is not a generalization gap; it is a units mismatch, and no arithmetic in this report crosses

between the two. (A related source conflict: `config.yaml` labels the competition metric `rmsle`, while the leaderboard value it is compared against is plainly in dollars. The two are not reconciled in the sources and are reported here as-is rather than harmonized.)

Why it matters. Ames Housing is the reference small-tabular regression problem: ~1,400 usable training rows against 80 mostly categorical features, with a right-skewed target and a well-known pair of outliers. There is no compute advantage to be had — everything hinges on target transformation, encoding choices, outlier handling, and ensemble diversity. It is the purest available test of whether an autonomous agent can build a disciplined pipeline rather than a large one.

Data

Purpose of Data. Predict the sale price of a residential property from its physical, locational, and transactional attributes. **Data Format** is **tabular CSV** with a `data_description.txt` companion; **Data Volume**, per `config.yaml`, is the Ames set with **80 features** covering lot, building, condition, utilities and sale information, and AIDE's own report characterizes the training set as roughly 1,400 rows — three orders of magnitude smaller than the Playground Series episodes in this batch.

Data Quality work is embedded entirely in code; as in every AIDE run there is no EDA stage and therefore no EDA artifact, no correlation table, and no train↔test shift check. What AIDE actually did is nonetheless substantive:

- **Outlier removal.** At step 5 it removed the two canonical Ames outliers — `GrLivArea > 4000` with `SalePrice < 300000` — explicitly to address the "fold instability" it had been observing. This is one half of the largest single improvement in the run (step 4 → step 5, 0.1215 → 0.11193). AIDE's plans mention a recurring weak fold repeatedly, and this was its response.
- **Missing values.** Categorical columns are filled with an explicit `"Missing"` level; numeric columns with the column median. Crucially, imputation is computed on the **concatenation of train and test** — a deliberate choice for encoding consistency that also means test-set feature distributions leak into the imputation statistics. On this dataset the effect is negligible, but it is a leak in principle and AIDE neither flagged nor tested it.
- **Target transformation.** `log1p(SalePrice)` from step 0 onward, with `expm1` inversion and a clip at zero at submission time. AIDE stated the justification correctly in its first plan — RMSLE on the raw target is RMSE on the log target — and never deviated.

Annotation Guidelines. The label is the continuous `SalePrice` in dollars, modelled throughout as `log1p(SalePrice)`. The submission is a **point prediction per `Id`**, inverse-transformed back to dollars.

Feature Set. The champion builds one engineered frame and then materializes it in three representations, one per learner family:

Group	Features
Area / age composites	TotalSF (basement + 1st + 2nd floor), HouseAge (YrSold - YearBuilt), RemodAge , TotalBath (full + ½ • half, above grade and basement)
Quality × area interactions	OverallQual_TotalSF , OverallQual_GrLivArea , GarageArea_Qual
Ordinal quality encodings	_ord versions of ExterQual , ExterCond , BsmtQual , BsmtCond , HeatingQC , KitchenQual , FireplaceQu , GarageQual , GarageCond , PoolQC (Ex/Gd/TA/Fa/Po → 5...1), plus BsmtExposure , BsmtFinType1 , BsmtFinType2 , GarageFinish , Functional , Fence , PavedDrive , LotShape
Target encoding	Neighborhood_TE — smoothed mean of log1p(SalePrice) per neighbourhood, smoothing constant 10, computed within-fold
Representation A (CatBoost)	all raw categoricals passed natively via cat_features
Representation B (XGBoost)	full one-hot expansion via get_dummies
Representation C (Ridge / ElasticNet)	one-hot expansion, plus log1p on skewed numerics (year columns excluded) and StandardScaler

The **ordinal encoding of quality features** (step 11) is the run's clearest piece of domain reasoning: AIDE correctly observed that one-hot encoding destroys the monotone ordering in Excellent > Good > Typical > Fair > Poor, and that both trees and the linear models could exploit it if preserved. The **within-fold Neighborhood target encoding** (step 8) is the run's clearest piece of methodological care — leakage-safe by construction inside the CV loop, and the agent said so in the plan.

Splitting strategy. KFold(n_splits=5, shuffle=True, random_state=42) — unstratified, which is correct for regression — fixed from step 0 so all 18 scored nodes are comparable. Three **inner 3-fold grid searches** sit inside the champion (CatBoost depth × l2_leaf_reg , Ridge alpha , ElasticNet alpha × l1_ratio), and the convex blend weights are evaluated through a **nested 5-fold meta-CV** on the OOF matrix, which is better hygiene than most of this batch. The competition is an evergreen Kaggle Learn exercise with a rolling public leaderboard and **no private leaderboard**, so facts_aide.json carries a public score only.

Models & Architecture

Purpose of Architecture. Minimize RMSE on log1p(SalePrice) . **Architecture Type** is a **convex-weighted blend of four heterogeneous base learners** — two gradient-boosted tree families and two regularized linear models — with weights fitted by constrained optimization rather than by a meta-model.

Input Format differs per learner (three representations above); **Input Dimension** is the engineered frame plus the target encoding: a few hundred columns after one-hot expansion for XGBoost/Ridge/

ElasticNet, and the compact native-categorical frame for CatBoost. All are 1-D feature vectors with no spatial or sequential structure.

Architecture Description.

- **XGBoost** — `objective="reg:squarederror"`, `eta=0.05`, `max_depth=4`, `subsample=0.8`, `colsample_bytree=0.8`, up to 2000 rounds with `early_stopping_rounds=50`, **seed-bagged over SEEDS = [42, 123, 7]** (three models averaged per fold).
- **CatBoost** — `iterations=2000`, `learning_rate=0.05`, `loss_function="RMSE"`, `early_stopping_rounds=50`, native categoricals, **seed-bagged over the same three seeds**, with `depth` and `l2_leaf_reg` selected by an inner 3-fold grid over `depth` $\in \{4, 6, 8\} \times$ `l2_leaf_reg` $\in \{3, 5, 10\}$.
- **Ridge** — on the scaled, skew-corrected one-hot frame, with `alpha` selected by inner 3-fold grid over $\{0.1, 1, 3, 5, 10, 20, 50, 100, 200\}$; AIDE's report records 200 — the strongest regularization on the grid — as the winner.
- **ElasticNet** — same input, `max_iter=20000`, with `alpha` $\in \{0.0005, 0.001, 0.005, 0.01\} \times$ `l1_ratio` $\in \{0.1, 0.5, 0.9\}$ by inner 3-fold grid; AIDE's report records `alpha=0.005`, `l1_ratio=0.9`.
- **Blender** — `scipy.optimize.minimize(method="SLSQP")` on the 4-column OOF matrix, minimizing OOF MSE subject to weights non-negative and summing to 1.

Model Complexity is dominated by the tree members: per fold, up to 2000 rounds $\times$ 3 seeds for each of XGBoost (depth 4) and CatBoost (tuned depth), truncated by early stopping, over 5 folds — several tens of thousands of trees in total, sitting under a blender with exactly four free parameters. AIDE's own report records that **CatBoost consistently drew the largest blend weight** and that **LightGBM and SVR were both driven to essentially zero weight** and eventually deleted from the ensemble (steps 13 and 18) — the convex constraint acting as automatic member selection, which is precisely what AIDE introduced it for at step 10.

Training procedures

The search trajectory

Of the **20 steps** in the budget, **18 produced a score and 2 crashed**:

$$20 - 18 = 2 \text{ buggy steps}$$

Both failures are `TimeoutError` (`buggy_exc_types: {"TimeoutError": 2}`), at the two steps absent from the metric trajectory — **steps 17 and 19**. Each hit the 1800 s harness wall (`exec_time_max_s = 1800.0 s`), so together they consumed

$$\begin{aligned} 2 \times 1800 &= 3600 \text{ s} \\ 3600 / 6098.8 &\approx 0.5903 \rightarrow 59.0\% \text{ of all execution time} \\ 6098.8 / 3600 &\approx 1.69 \text{ h total} \end{aligned}$$

Nearly 60% of this run's compute produced nothing — the worst waste ratio in the batch, and on the smallest dataset in it. The cause is cumulative rather than sudden: by step 14 the pipeline already carried three sequential inner grid searches plus seed-bagged 2000-round boosters inside a 5-fold loop, and the per-node cost had climbed steadily (mean **304.9 s** across all 20 nodes). Step 17 then tried to add a fourth tuned base learner (LightGBM replacing SVR) on top of that, and step 19 — the final step — tried to run the entire OOF-generation loop **twice**, under two different `KFold` seeds, to damp the fold instability AIDE had been complaining about since step 5. Both exceeded the box. The second of these is the more costly mistake: the last step of the budget was spent on a doubling of the most expensive part of the pipeline, with no fallback, so the run ended on a crash rather than on a consolidation.

The trajectory is otherwise the healthiest in this batch — a real descent, not a plateau (lower is better):

Step	What AIDE changed	CV RMSLE (log scale)
0	LightGBM, label-encoded categoricals, log target (baseline)	0.12956
1	XGBoost, one-hot encoding	0.12769
2	XGBoost + LightGBM equal-weight average	0.12712
3	+ feature engineering (<code>TotalSF</code> , <code>HouseAge</code> , <code>TotalBath</code> , <code>quality × area</code>)	0.12575
4	+ Ridge as a third, non-tree member	0.1215
5	stacking with a Ridge meta-learner + removal of the two Ames outliers	0.11193
6	+ CatBoost as a fourth base learner	0.11181
7	standalone PyTorch MLP	0.47989
8	+ within-fold <code>Neighborhood</code> target encoding	0.11173
9	feature-weighted linear stacking (context features in the meta-input)	0.11215
10	convex-weight blender (SLSQP) replaces the Ridge meta-learner	0.11155
11	+ ordinal encoding of quality/condition features	0.1116
12	+ seed-bagging of the tree members	0.11132
13	SVR replaces the zero-weighted LightGBM	0.11117
14	CatBoost <code>depth</code> / <code>l2_leaf_reg</code> grid search	0.11105
15	XGBoost grid search, SVR dropped	0.11118
16	Ridge <code>alpha</code> grid search	0.11087
17	tuned LightGBM replaces SVR	TimeoutError
18	ElasticNet replaces SVR (final)	0.11031
19	repeated K-fold (two seeds, full loop run twice)	TimeoutError

Total improvement over AIDE's own first solution:

$0.12956 - 0.11031 = 0.01925$
 $0.01925 / 0.12956 \approx 0.1486 \rightarrow 14.9\% \text{ relative reduction in RMSLE}$

The descent has clear structure. The first five steps are broad exploration, and step 5 is the inflection — stacking plus outlier removal together deliver more than those five steps combined:

```
0.12956 - 0.1215 = 0.00806    (steps 0 → 4, broad exploration)
0.1215 - 0.11193 = 0.00957    (step 5 alone: stacking + outlier removal)
```

and it is the removal of two rows, not the meta-learner, that AIDE's own analysis credits. Everything after step 5 is refinement inside a narrow band:

```
0.11193 - 0.11031 = 0.00162    across the final 13 steps
0.11087 - 0.11031 = 0.00056    largest late gain (step 16 → step 18)
```

Within that band the moves are nonetheless real and mostly monotone — convex blending (step 10), seed bagging (step 12), and three sequential hyperparameter searches (steps 14–16) each shaved a little, and the final member swap at step 18 gave the largest late gain of all. Setting the MLP aside, only three nodes regress against their immediate predecessor: step 9's feature-weighted stacking (0.11215 against step 8's 0.11173), step 11's ordinal encoding (0.1116, a hair above step 10's 0.11155), and step 15's XGBoost tuning with SVR dropped (0.11118 against step 14's 0.11105). Notably AIDE kept the ordinal features anyway — they persist into the champion script — which is the right call for a difference of five hundred-thousandths on ~1,400 rows, but it was a judgement call the agent made silently rather than a tested one.

Step 7 is the run's outlier in every sense: a standalone PyTorch MLP scoring

```
0.47989 - 0.11181 = 0.36808    worse than the incumbent
```

on a ~1,400-row, high-dimensional sparse one-hot input — a near-mean predictor. AIDE diagnosed it correctly in its report (dataset too small, input too sparse, insufficient regularization) and, importantly, **abandoned the axis immediately** rather than trying to rescue it, which is the right call and is not what it did with some cheaper dead ends elsewhere.

One artifact discrepancy must be surfaced. The `leaderboard` block in `facts_aide.json` labels the submission "20-step best node, local 0.47989" — the step-7 MLP score — whereas `best_metric` and `score_table.aide_local` both record **0.11031**. The label is a direction bug: this is the one minimize-direction competition in the batch, and the message string appears to have selected the **maximum** metric rather than the minimum, whereas the score table applied `direction: min` correctly. Inspection of the submitted `working/submission.csv` supports the score table — the predicted price distribution is realistically dispersed across the plausible Ames range rather than near-constant, which is not what a 0.47989-RMSLE model produces, and a mean-predicting model could not have placed at the 98.3rd percentile. The submitted artifact is the real champion; only the message label is wrong.

Training configuration

The **Loss Function** is squared error on the log target for every member — `reg:squarederror` for XGBoost, `RMSE` for CatBoost, and the ordinary least-squares objective plus an L2 (Ridge) or L1+L2

(ElasticNet) penalty for the linear members — which is exactly the right objective, since `RMSE on  $\log_{1p}(\text{SalePrice})$` is the competition metric. The blender minimizes the same quantity (OOF MSE) under a simplex constraint. The **Optimization Algorithm** is threefold: gradient boosting for the tree members (**not** SGD/ADAM), closed-form / coordinate-descent solvers for Ridge and ElasticNet, and **SLSQP** (sequential least-squares quadratic programming) for the four blend weights. The **Learning Rate** is the boosting shrinkage, 0.05 for both XGBoost (`eta`) and CatBoost, fixed throughout and never tuned; the linear members have no learning rate, only regularization strengths (`alpha`, `l1_ratio`), which were tuned. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by per-fold early stopping at 50 rounds, not a schedule; the linear solvers converge to an analytic or coordinate-descent optimum) and **no Batch Size** (N/A — full-dataset boosting and full-batch linear fitting, not mini-batched; the single mini-batched model in the run, the step-7 MLP, scored 0.47989 and was discarded).

Training Duration across the whole search was **6,098.8 s** over 20 nodes, mean **304.9 s**, max **1,800.0 s** (the harness cap). Per-node wall time for the champion is not carried in `facts_aide.json` — **not recorded**. **Training Memory** was never instrumented — **not recorded**; a ~1,400-row frame, even one-hot expanded, is trivial, and the cost here is entirely CPU time spent on three seed-bagged 2000-round boosters inside nested grid searches.

Transfer Learning is N/A (no pretrained weights). AIDE has **no cross-competition memory** — every run starts cold — and its analogue is the within-run journal, which is unusually visible in this run: plans at steps 12, 13, 14, 15, 16 and 19 all reason from accumulated observations ("LightGBM has consistently received near-zero weight in the convex blender", "CatBoost has consistently received the highest blend weight (~0.40– 0.56)", "the recurring fold instability noted throughout the memory"). This is the one competition where AIDE's search demonstrably learned about its own ensemble across steps rather than merely hill-climbing a scalar. **Data Augmentation** is N/A (tabular); the analogues here are feature engineering (steps 3, 8, 11 — all retained) and seed bagging (step 12 — retained, and measurable):

```
0.1116 - 0.11132 = 0.00028    (seed bagging, step 11 → step 12)
```

Reproducibility Standards are the best of this batch but still incomplete. `RANDOM_STATE = 42` is applied consistently across the outer `KFold`, all three inner tuning `KFold`s, the meta-CV, Ridge, ElasticNet, and `random_seed` for the non-bagged paths, with `SEEDS = [42, 123, 7]` fixed for bagging. No determinism flags or thread pinning are set for XGBoost or CatBoost, so bit-identical reproduction across differing hardware is not guaranteed, and there is **no rebuild or reproduction gate** — AIDE never retrains its champion to verify the cached metric.

Two leakage caveats belong here rather than being left implicit. First, imputation statistics (categorical "Missing" fill, numeric medians) are computed on the **train+test concatenation**, so test feature distributions inform the training frame. Second, and more consequentially: inside the CV loop the `Neighborhood` target encoding is fold-safe, but the **final refit block re-encodes the training rows using the full-training target** before fitting the models that actually produce the submission. The reported 0.11031 is therefore leak-free, but the shipped estimator is not built the same way the

measured one was. A third, milder issue: the final XGBoost and CatBoost refits select their round counts from **a single arbitrary fold split** (`next(KFold(...).split(...))`) rather than from the CV-average best iteration.

Inference procedures

Decision Threshold is N/A — this is a regression task; there is no thresholding, and no rounding or snap-to-grid post-processing of the kind price competitions sometimes reward.

Post-processing is a two-step inversion: the convex-weighted blend is formed on the log scale, then `np.expml` returns dollars, then `np.clip(..., 0, None)` floors the result at zero to guard against a negative prediction surviving the inversion. The submission is written as `Id, SalePrice` against `test_ids` captured before preprocessing, preserving the original row order.

The **shipped estimator is not the measured one**, and the difference matters more here than elsewhere in this batch. The reported 0.11031 comes from a nested 5-fold convex blender over fold-trained base models; the submission comes from base models **refit on the full training set** (with round counts borrowed from one fold) and blended with weights fitted on the **full OOF matrix**. AIDE's own report also notes, in the same row as the champion, that the simple-average blend of the same four members scored better on CV than the convex blend it actually submitted — that figure is not carried in `facts_aide.json` and so is not quoted here, but the direction is worth stating: **the search's final model-selection step may have chosen the worse of two blends it had already computed**. **Inference Duration** and **Inference Memory** were not profiled — **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict a dollar sale price for each test house.

Performance Metrics — stated in two incompatible units.

Quantity	Value	Units
AIDE local CV (<code>aide_local, best_metric</code>)	0.11031	RMSE on <code>log1p(SalePrice)</code> — dimensionless
AIDE public leaderboard (<code>aide_pub</code>)	12887.15456	raw-dollar error scale
Public leaderboard rank	72 / 4103	—
Public leaderboard percentile	98.3	—

These two scores cannot be differenced, ratioed, or plotted on one axis. The local figure is what the 20-step search optimized and is the only number the trajectory table above is measured in; the leaderboard figure is what Kaggle returned, on the untransformed price scale. Every piece of arithmetic in this report stays inside one unit system. The competition has no private leaderboard, so no public→private shift can be examined.

Within the local (log-scale) system, the search delivered:

```
0.12956 - 0.11031 = 0.01925
0.01925 / 0.12956 ≈ 0.1486 → 14.9% relative reduction
0.1215 - 0.11193 = 0.00957 → half the total, from step 5 alone
```

Half of the whole descent arrived in the single step that removed two outlier rows and introduced stacking.

Performance Benchmarking. `facts_aide.json` 's score table records `local_winner: aide` and `lb_winner: aide (pub)` — AIDE won this competition against both comparator lanes — but it carries **no mine_priv or nvidia_priv values** for home-data-for-ml-course, so a numeric three-way table cannot be built and none is invented:

Agent	Approach	Public LB (raw-dollar scale)	Note
AIDE	20-step search → XGB + CatBoost + Ridge + ElasticNet convex blend	12887.15456	rank 72 / 4103, 98.3rd pct; recorded winner
From-scratch agent	not carried in <code>facts_aide.json</code>	—	recorded as losing both comparisons
NVIDIA	not carried in <code>facts_aide.json</code>	—	no figure available

Three honest qualifications. First, **the 98.3rd percentile flatters the result**: home-data-for-ml-course is an evergreen Kaggle Learn exercise whose 4,103-entry leaderboard is dominated by tutorial submissions, so a percentile here is not comparable to the same percentile on a competitive, time-boxed Playground Series episode, where the field is smaller and uniformly stronger. Second, **the pipeline that was measured is not the pipeline that was submitted** (full-data target re-encoding, single-fold round selection, and blend weights fitted on the full OOF matrix), and AIDE's own report suggests its final blend choice may have been the worse of two available options. Third, **59.0% of the run's compute was lost to two self-inflicted timeouts**, both from adding cost to an already-expensive pipeline with no fallback — including the very last step of the budget, which the agent spent doubling its most expensive loop and consequently ended on a crash.

Set against that, this is the run in which AIDE behaved most like a competent data scientist: it transformed the target correctly on the first try, found and removed the canonical outliers, reasoned about ordinal structure in quality features, built a leakage-safe within-fold target encoding, replaced an overfitting meta-learner with a constrained blend, used that constraint to prune two redundant members, and tuned the members in order of their blend weight. The descent from 0.12956 to 0.11031 is earned, step by step, and with only three small regressions in eighteen scored nodes it is the closest thing to a monotone improvement curve in this batch.